Dokumentacja Poker

Baza danych

Tabela Users (Konta graczy i uprawnienia)

Kolumna	Typ	Ograniczenia
id	INT	PRIMARY KEY AUTOINCREMENT
username	TEXT	NOT NULL UNIQUE
email	TEXT	NOT NULL UNIQUE
Password_hash	TEXT	NOT NULL
role	TEXT	NOT NULL DEFAULT 'user'
chips	INT	NOT NULL DEFAULT 10000
Avatar_url	TEXT	NULL
bio	TEXT	NULL
Is_banned	INT	NOT NULL DEFAULT 0
Lockout_until	TEXT / DATETIME	NULL
Lockout_reason	TEXT	NULL
Last_bonus_at	TEXT / DATETIME	NULL
Created_at	TEXT / DATETIME	NOT NULL DEFAULT CURRENT_TIMESTAMP
Updated_at	TEXT / DATETIME	NOT NULL DEFAULT CURRENT_TIMESTAMP

Tabela refresh_tokens (Zarządzanie sesjami i wylogowaniem)

Kolumna	Typ	Ograniczenia
id	INT	PRIMARY KEY AUTOINCREMENT
User_id	INT	NOT NULL, FK -> users(id) ON DELETE CASCADE
Token_hash	TEXT	NOT NULL UNIQUE
Expires_at	TEXT /DATETIME	NOT NULL

Created_at	TEXT /DATETIME	NOT NULL DEFAULT CURRENT_TIMESTAMP

Tabela chip_transactions (Historia operacji na żetonach)

Kolumna	Typ	Ograniczenia
id	INT	PRIMARY KEY AUTOINCREMENT
User_id	INT	NOT NULL, FK -> users(id)
amount	INT	NOT NULL
type	TEXT	NOT NULL
description	TEXT	NULL
Created_at	TEXT / DATETIME	NOT NULL DEFAULT CURRENT_TIMESTAMP

Backend

Miłosz Szymczuk

- Struktura backendu w Express (routing, kontrolery, middleware).
- Obsługa błędów i odpowiedzi 404 w jednolitym formacie.
- Walidacja danych wejściowych.
- Projekt i utworzenie bazy (schemat + dane przykładowe).
- Warstwa repozytoriów – wszystkie zapytania do bazy.
- Przejście endpointów z pamięci na bazę danych.
- Zapis zakończonych rozdań przekazanych przez silnik gry.
- Testy API dla najważniejszych przypadków.

Miłosz Pawlaczyk

Silnik Gry:
Deck, Table, HandEvaluator, PotManager, licytacja, timer 30s

Warstwa WebSocket / Socket.IO dla rozgrywki:

- Obsługa zdarzeń stołu (lobby)- table:join, table:leave, game:action
- Rozsyłanie - table:state, game:holeCards, game:turn, game:showdown
- Obsługa zdarzeń czatu w lobby – chat:send, chat:message, game:handEnded

Endpointy zajmowania/opuszczania miejsca przy stole oraz zapis rozdań do bazy:

- POST /api/v1/tables/:id/seats (*buy-in* – zajęcie miejsca przy stole i pobranie żetonów z salda)
- DELETE /api/v1/tables/:id/seats/me (*cash-out* – odejście od stołu i rozliczenie stacka w bazie)
- GET /api/v1/hands, GET /api/v1/hands/:id (pobieranie historii rozdań i szczegółów rozdania z akcjami).
- GET /api/v1/tables/:id/messages?limit=50 (pobieranie ostatnich wiadomości czatu po wejściu do stołu

Testy jednostkowe silnika:

- testowanie podziału *side potów* przy *all-in* i remisów w Node:test / Vitest

Michał Małecki (Auth, Security, API Admina, Profile):

Autoryzacja i profil

- POST /api/v1/auth/register, POST /api/v1/auth/login – rejestracja i logowanie (wraz z weryfikacją i generowaniem tokenów JWT)
- POST /api/v1/auth/refresh, POST /api/v1/auth/logout – obsługa odświeżania sesji w tle i wylogowania
- GET /api/v1/users/:id, GET /api/v1/users/:id/stats – pobieranie danych do widoku profilu oraz obliczanie statystyk gracza
- POST /api/v1/users/me/bonus – weryfikacja i odbiór dziennego bonusu (dopisanie żetonów do salda)
- POST /api/v1/auth/change-password – zmiana hasła z walidacją aktualnego
- GET /api/v1/auth/me – pobieranie danych aktualnie zalogowanego użytkownika
- PATCH /api/v1/users/me – edycja własnego profilu (aktualizacja bio, zmiana awatara)

Panel Administratora

- GET /api/v1/admin/users – pobieranie listy wszystkich użytkowników

- PATCH /api/v1/admin/users/:id – nakładanie blokady na konto (ban) oraz zmiana ról (user / admin)
- POST /api/v1/admin/users/:id/chips – ręczne przyznanie wirtualnych żetonów wybranemu graczowi

Warstwa bezpieczeństwa i Middleware

- Zabezpieczenie bazy danych (haszowanie haseł za pomocą bcrypt) oraz uwierzytelnianie połączeń (JWT w ciasteczkach)
- Zabezpieczenie API przed atakami: walidacja zapytań przez express-validator (ochrona przed przypisywaniem sobie ról/żetonów przez graczy), wdrożenie helmet, cors oraz express-rate-limit (limitowanie logowań)
- Uwierzytelnianie połączeń: Access Token w pamięci aplikacji, Refresh Token w bezpiecznym ciasteczku HttpOnly (SameSite: Strict).
- authenticateJWT – middleware weryfikujący token i blokujący dostęp niezalogowanym użytkownikom.
- requireRole('admin') – middleware kontroli dostępu (RBAC) chroniący endpointy /api/v1/admin/*.
- checkAccountStatus – middleware sprawdzający przed wykonaniem akcji, czy konto nie jest aktywnie zablokowane (lockout_until).

Frontend

Maciej Cyuńczyk:

- TableView.vue (Stół gry)
- PlayingCard.vue (karty)
- TableCanvas.vue, SeatSpot.vue
- ActionBar.vue (licytacja)

- HandDetailsView.vue (Replayer)

Michał Małecki (Auth, Security, API Admina, Profile):

- LoginView.vue
- RegisterView.vue
- AdminView.vue
- ProfileView.vue

Michał Pączkowski:

Home.vue

ResultOverlay.vue

LeaderboardView.vue

Animacje + Dźwięk

HandHistory.vue

Miłosz Pawlaczyk:

- LobbyView.vue

Przebieg startu gry

Użytkownik tworzy stół (podaje ustawienia stołu : czas podjęcia decyzji, maksymalna pula startowa punktów, mała stawka, duża stawka).

Host czeka na resztę graczy.

Reszta użytkowników dołączają do stołu.

Możliwość kliknięcia gotowości do gry.

Jeżeli wszyscy są gotowi to gra zaczyna odliczanie do startu gry.

Gdy większość graczy jest gotowa zaczyna się obliczanie do kliknięcia gotowości. Jeżeli gotowość nie zostanie kliknięta przez gracza zostaje on wyrzucony z lobby.

Przebieg gry wyglada tak jak w grze poker Texas Hold’em

Losuje pierwszego gracza

Pierwszy gracz automatycznie stawia małą stawkę

Kolejni gracze mają do wyboru pass, podbicie lub dobicie do dużej stawki